PROJECT PRESENTATION & GUIDE EXPLANATION MANUAL

Air Quality Analysis using Apache Spark and PySpark

A Step-by-Step Script and Viva Defense Guide for Review with Faculty Guide

---

1. The 2-Minute Elevator Pitch (Opening Statement)

When your guide asks: *'Give me a summary of your project and what problem it solves'*, deliver this exact structured response:

What to say verbatim:

*"Respected Guide, our final-year Big Data project is titled **Air Quality Analysis using Apache Spark and PySpark**. In this project, we analyze multi-year environmental time-series data across five major metropolitan Indian cities: **Bengaluru, Chennai, Delhi, Hyderabad, and Mumbai**, comprising over 12,000 historical records from 2018 to 2024.*

*The core engineering challenge in environmental monitoring is handling **disparate, ragged schemas, inconsistent date formats, and missing pollutant readings**. Rather than relying on single-threaded libraries like standard Pandas, we engineered an end-to-end distributed data pipeline using **Apache Spark 3.5.6** running in multi-threaded local mode. Spark enforces strict 9-column schemas, cleans date discrepancies, and performs statistical **per-city median imputation** using Spark's percentile_approx function.*

*On top of the PySpark aggregation engine, we built an interactive, reactive web dashboard using **Streamlit and Plotly**. The dashboard enables dynamic multi-city filtering, date range slicing, real-time KPI scorecards, particulate correlation analysis (PM2.5 vs PM10), and export of filtered subsets. Everything runs entirely locally on our laptop without needing heavy cloud credentials or a physical Hadoop cluster."*

2. Step-by-Step Live Demonstration Walkthrough

Follow these 4 chronological stages while presenting on screen:

Stage 1: Raw Data Ingestion & Challenges (data/ folder)

- **Show the folder:** Open C:\AirQualityAnalysis\data\ containing the 5 CSVs.
  - **Explain the challenge:** Mention that raw CSVs contained uneven columns, trailing commas, and mixed date styles (dd/MM/yy vs dd/MM/yyyy).
  - **Run command:** python inspect_data.py to show Spark successfully loading schemas with master('local[*]').

Stage 2: Distributed PySpark Data Cleaning Pipeline (clean_data.py)

- **Open clean_data.py in editor:** Highlight the StructType definition for strict 9 columns (City, Date, AQI, PM2.5, PM10, NO2, SO2, CO, O3).
  - **Highlight City Normalization:** Show F.when(lower(trim(City)) == 'bangalore', 'Bengaluru').
  - **Highlight Date Parsing:** Show F.coalesce() across three date format patterns.
  - **Highlight Median Imputation:** Show F.percentile_approx(col, 0.5) grouped by City. Explain that median was chosen over mean because environmental pollution has extreme outlier events.
  - **Explain Windows Compatibility:** Mention that the cleaned dataset is exported via Pandas to output/cleaned_air_quality.csv to bypass Windows-specific winutils.exe limitations while preserving Spark's distributed processing.

Stage 3: Spark Distributed Aggregation & Insights (analysis.py)

- **Run command in terminal:** python analysis.py
  - **Point to the terminal output:**
    - **Total Records:** 12,042 observations across 5 cities spanning 2018 to 2024.
    - **City Comparison:** Delhi has the highest average AQI (208.29), followed by Mumbai (106.78), Hyderabad (85.40), Chennai (77.33), and Bengaluru (74.32).
    - **Particulate Matter:** Delhi averages 114.56 PM2.5 and 224.95 PM10, nearly 3x higher than Bengaluru (40.88 PM2.5).
    - **Top 10 Extreme Days:** All top 10 peak AQI days occurred in Delhi (peaking at 494.0 in Nov 2019 and Nov 2024), demonstrating intense winter seasonal smog.

Stage 4: Interactive Web UI (app.py at http://localhost:8501)

- **Run command:** streamlit run app.py (or switch to the browser window).
  - **Demonstrate Sidebar Filters:** Select 'Delhi' and 'Bengaluru' together; observe how KPI cards and charts dynamically update instantly.
  - **Demonstrate Date Slider:** Filter down to year 2023–2024 to show responsive query updates.
  - **Walk through the 5 Tabs:**
    1. *City Comparison:* Bar chart of average AQI and Box plots of outlier spreads.
    2. *Yearly & Monthly Trends:* Multi-city line chart illustrating the winter smog peak (Nov–Dec).
    3. *Particulate & Pollutants:* Scatter plot proving high correlation between PM2.5 and PM10.
    4. *Extreme Events:* Horizontal ranking chart of top 10 worst pollution days.
    5. *Data Explorer:* Live searchable preview with a 1-click 'Download Filtered Data as CSV' button.

3. Core Big Data Concepts You Must Emphasize

Guides look for theoretical foundation and mastery of Big Data concepts. Use these exact technical keywords:

Concept	How to Explain to Guide
Why PySpark over Pandas?	Pandas executes single-threaded on a single CPU core and is bounded by physical RAM. Apache Spark operates in-memory with parallel partitions across all available cores (local[*]), supporting horizontal scaling for massive multi-gigabyte or streaming sensor datasets.
Lazy Evaluation & Catalyst Optimizer	Spark does not execute transformations (filter, groupBy, withColumn) immediately. Instead, it constructs a Directed Acyclic Graph (DAG). The Catalyst Optimizer reorganizes expressions and filters before an action (count, show) triggers execution.
Percentile Imputation (percentile_approx)	Rather than filling missing readings with arbitrary global constants or simple arithmetic means, we used PySpark's percentile_approx(col, 0.5) grouped by City. Median imputation is robust to environmental outliers and preserves regional baseline characteristics.
Caching Strategy (@st.cache_data & df.cache())	In Spark, df.cache() prevents re-evaluating the upstream DAG across multiple analytical queries. In Streamlit, @st.cache_data stores the parsed dataframe in memory to ensure sub-second UI responsiveness.
Column Escaping (Backticks)	In Spark SQL, periods represent nested struct fields. Because columns like PM2.5 include a dot, PySpark requires backticks (F.col(`PM2.5`)) to reference the literal column name safely.

4. Anticipated Guide Questions & Ready-to-Answer Responses

Prepare for these common cross-examination questions during your evaluation:

Q1: Why did you run Spark in local mode instead of deploying on a physical Hadoop cluster?
Local mode (local[*]) executes the complete Spark engine with multi-threaded parallel executors across all laptop CPU cores. It provides full PySpark API fidelity and Catalyst optimization without the hardware, network, and operational overhead of maintaining a multi-node YARN/HDFS cluster for development and evaluation.

Q2: Why did you use Pandas to write the CSV file at the end of clean_data.py?
On native Windows machines without Hadoop binaries, Spark's native Hadoop FileSystem writer throws a winutils.exe file permission exception. Exporting the consolidated partition via Pandas cleanly bypasses this Windows OS limitation while preserving all distributed cleaning operations in Spark.

Q3: Does this dashboard display live or real-time air quality?
No, respected Guide. This is explicitly a historical time-series analytics project analyzing 12,042 validated records from 2018 to 2024. We have clearly documented this limitation in the dashboard footer and README to maintain academic and data-integrity rigor.

Q4: How would you extend this project into an enterprise production system?
We would ingest live sensor telemetry streams using Apache Kafka, process them in real-time via Spark Structured Streaming, train AQI forecasting models using PySpark MLlib (Random Forest Regressors), and persist data into a Delta Lake on cloud object storage (AWS S3 / GCP Cloud Storage).

5. Critical Project Statistics to Memorize

Metric	Exact Value	Context / Explanation
Total Cleaned Records	**12,042**	Bengaluru: 2556, Hyderabad: 2556, Chennai: 2551, Delhi: 2191, Mumbai: 2188
Date Span	**2018 to 2024**	7 full calendar years of observations
Delhi Average AQI	**208.29**	Highest in India; Maximum observed AQI reached **494.0**
Bengaluru Average AQI	**74.32**	Lowest / cleanest among the 5 metropolitan cities
Dominant Pollutant	**PM2.5 & PM10**	Particulate matter drives >85% of elevated AQI episodes